

摘要

及时排查清除干扰源是无线电频谱管理的重要任务。干扰源的传统精准定位需技术人员徒步排查、定位并清除，耗时费力且存在安全风险，机器狗的发展为该问题提供了新的解决途径。本文针对利用多个检测点示向度对干扰源进行无源定位，并由机器狗在频道、位置与有效接收半径均未知的条件下快速自主完成发现、定位与彻底清除的问题，建立了从几何交会定位到路径调度优化的完整模型体系。

针对问题一：针对示向误差不超过 1° 的凸锥半平面模型，将每个示向度转化为两条半平面约束，利用 Sutherland-Hodgman 顺序裁剪求交，并以旋转卡壳算法在 $O(n \log n)$ 时间内计算定位区域直径。通过内接、外切正 720 边形对目标圆域双侧逼近，直径上下界差在 10^{-13} m 量级。证明直径圆覆盖的充要条件为最小覆盖圆半径等于直径之半；等边三角形反例表明直径圆一般不能覆盖，Jung 定理给出保证半径上界 $D/\sqrt{3}$ 。数值算例定位区域为六边形，直径 44.14 m，直径圆恰好覆盖，验证了算法可靠性。

针对问题二：在首次示向度已知条件下，定义完整可能源集合 U_1 ，构建 1000 m 保证接收区 C_{1000} 与利用首次成功检测信息的条件保证区 C_{det} 。采用一阶误差传播构造面积代理 $A_{\text{proxy}} = \pi F_1^2$ ，通过 Pareto 非支配筛选与字典序选点确定第二检测点。验证场景中选得 $S_2 = (993.74, -143.47)$ m，局部耗时 205.81 s，最坏确定性误差界 60.88 m，保证最远源距 999.91 m。该策略兼顾接收可靠性与定位精度，避免仅优化交会角的片面性。

针对问题三：设计七点扫描骨架（原点加半径 1200 m 正六边形），证明最坏最近点距离 968.90 m < 1000 m，可发现全部全向源。提出保证第二观测点 $S' = S + 400u + 300v$ ，其接收信号有严格证明；以最小覆盖圆半径 ≤ 20 m 为清除判据，并设网格兜底。两阶段批量调度将平均定位清除时间从 507.9 s 降至 418.5 s（降 17.6%）。25 局官方演练清除达标 25/25，提速版 15 局平均 262.2 s，清除失败 0 次。三次正式测试清除干扰源 40 个，平均定位清除时间 289.91 s，程序运行时间 2198–2404 ms，空清除 0 次，20 个频道全部取得确定性结论。时间构成中移动占 80%，优化重点在路径。

针对问题四：引入定向源盲区导致“无信号”三义性，建立方向覆盖证书：扫描集须满足 $G \in \text{conv}\{P_j : \|P_j - G\| \leq 1000\}$ 。采用 31 点等边三角格（边长 990 m）或 22 点认证集，配合连续域凸包认证，保证任意未知朝向至少一个扫描点可见。定位遇无信号时转扇形 20 m 网格覆盖式清除，有限步内必成功。6 局演练清除 100%，提速版 15 局平均 499.1 s，清除 198/198；三次正式测试清除 39 个，平均 462.66 s，空清除 2 次均在预算内，程序运行时间 1976–2426 ms。模型以完备性优先，时间约为问题三的 1.7–2 倍，但程序运行时间远低于 1200 s 上限。

关键词： 无源测向定位；凸锥半平面交会；方向覆盖证书；覆盖式清除；路径调度优化

一、问题重述与分析

1.1 问题背景

无线电频谱的有序使用关系到通信、导航和广播等业务的正常运行。传统干扰排查通常经历初步监测、区域排查和人工近场定位，最后一环耗时且可能使人员暴露于危险环境。搭载测向机、光学定位装置和激光清除装置的四足机器人，能够把这一过程改写为“感知—决策—移动—清除”的自主闭环；但示向误差、未知接收半径、未知频道以及定向源盲区，使系统同时面临几何不确定性和在线路径决策问题[?]。

1.2 问题重述

问题一：由若干检测点坐标和同一干扰源的示向度，在 $\pm 1^\circ$ 误差下构造交会定位区域，求其直径，并判断以一组直径端点为直径的圆能否覆盖该区域。

问题二：已知某全向干扰源在第一检测点的示向度，设计第二检测点，使再次接收具有确定性保证，并在定位精度与移动检测耗时之间取得权衡，给出候选区域及推荐站址。

问题三：目标圆域内有10–16个频道互异的全向干扰源，其数量、位置、频道和接收半径均未知；机器狗从原点、频道1出发，自主完成发现、定位、清除与不存在认证，并最小化虚拟总时间。

问题四：在问题三中加入数量和朝向均未知的定向干扰源。此时一次无信号还可能由定向盲区造成，需重新设计发现证书与清除策略，并完成模拟器测试。

1.3 问题分析与总体思路

四问共享“**不确定集合—确定性证书—代价最小化**”主线。问题一把每个示向度写成闭凸测向锥，以半平面交获得定位集合，再用凸包与旋转卡壳求直径，并借最小覆盖圆和Jung定理回答覆盖性。问题二先由首次正检测构造完整可能源集合，再求保证接收的第二站候选区；以误差盒经Jacobian传播所得的最坏位置误差和局部耗时组成Pareto双目标，第二次读数返回后仍调用问题一的精确集合与最小覆盖圆作清除判定。

问题三把“存在源的定位清除”和“无源频道的完备认证”同时纳入在线路径：扫描站承担发现与阴性证据积累，多站示向度顺路交会，未达清除证书的频道才补充观测。问题四的关键不再是普通距离覆盖，而是使任意可能源附近的扫描点从各个半平面方向包围该源；只有这种方向完备性成立，连续无信号才足以排除未知朝向的定向源。最终以公开接口的动作日志复核清除闭合性、时间账和几何证书。

二、模型假设与符号说明

2.1 模型假设

题面明确给出的区域尺度、设备阈值和动作耗时均作为已知条件，不再改写为假设。全文采用以下统一假设。

- **假设1（静态平面几何）：**一次任务内干扰源位置保持不变，检测点坐标已知且执行无误差，按题面忽略测向机与干扰源的高度差（依据：题面条款与定位理想化；用于问题一至问题四）。

- **假设2（目标属性稳定）：**同一频道的有效观测对应同一干扰源，且其有效接收半径和定向方向在一局内不变（依据：题面条款与任务时段稳定性；用于问题一至问题四）。

- **假设 3 (确定性误差)**: 示向误差只满足区间界, 不预设分布或独立性; 同地重复检测的误差固定, 故不以重复平均缩小误差 (依据: 题面条款与最坏情形口径; 用于问题一至问题四)。
- **假设 4 (理想信号传播)**: 除题面给出的接收距离和定向覆盖外, 不另引入遮挡、多径或源间干扰 (依据: 模型闭合需要; 用于问题一至问题四)。
- **假设 5 (理想运动)**: 机器狗视为可即时转向并准确到达指令坐标的质点, 只计直线移动和题面规定的串行动作耗时 (依据: 题面运动规则与时间建模口径; 用于问题二至问题四)。
- **假设 6 (异常与退化)**: 约束交集为空表示观测不相容而非零面积; 区域退化为单点或线段时, 直径分别按零或线段长度定义 (依据: 连续集合定义与数值稳健性; 用于问题一至问题四)。

2.2 题面条件与计算口径

表 1 四问共用的题面条件

条件	数值	使用口径
目标圆域与源数	$R_0 = 1800\text{ m}$, $10 \leq N_J \leq 16$	频道为 1–20, 每频道至多一个源
示向误差	$ e_i \leq \delta = 1^\circ$	无分布; 保证性结论取确定性最坏情形
有效接收半径	1000–1500 m	保证接收下限; 保证超距无信号用上限
定向覆盖	中心方向两侧各 90°	取含边界的闭半平面, 朝向一局内固定
距离阈值	近距 5 m; 清除 20 m	近距可直接清除; 清除成功与源朝向无关
动作耗时	移动 5 m/s, 换频 1 s, 检测 5 s	光学定位 3 s, 激光清除 2 s, 动作串行

数学推导始终取题面界 $\delta = 1^\circ$; 问题三、四的在线程序另以 1.01° 包围示向度, 只用新增的 0.01° 吸收两位小数表示和浮点量化误差, 不改变物理模型。正常示向度意味着 $5 < \|G - S_i\|_2 \leq R^{\text{recv}} \leq 1500\text{ m}$; 首次正检测还给出 $R^{\text{recv}} \geq \max\{1000, \|G - S_1\|_2\}$, 问题二据此扩大保证接收区。

问题三中, 未清除频道的 `no_signal` 可能表示无源或超距; 问题四还可能表示定向盲区, 只有方向完备的扫描证书才能消除这种三义性。若已发现 16 个异频源, 可由数量上界认证其余频道无源; 否则任务仅在 20 个频道均为“已清除”或“确认不存在”时闭合。程序只串行调用 `/enter`、`/measure`、`/clear`、`/exit`, 幂等重试复用同一 `request_id`, 不读取案例真值。

几何程序把全部法向量单位化并采用相对尺度容差。问题一以正 720 边形内外包围圆域并报告直径上下界; 在线清除使用保守外包围。保证接收、超距判定和认证清除均向安全侧留数值余量; 这些阈值属于实现细节, 在相应算法章节给出, 不作为新的物理参数。

2.3 主要符号

表 2 问题一、二的主要符号

符号	含义	单位或取值	首次出现
S_i, G	第 i 个检测点与单个干扰源的位置向量	m	问题一
$\hat{\theta}_i, \theta_i, e_i$	示向度、真实方位角及示向误差	($^\circ$)	问题一
δ	题面示向误差半角	1°	问题一
$d(\phi)$	方位角 ϕ 对应的单位方向向量	无量纲	问题一
W_i, K	第 i 个测向锥与精确定位区域	集合	问题一
K_q^-, K_q^+	圆域内、外接正 q 边形所得定位区域	集合	问题一
$D(K), R_{\text{MEC}}(K)$	区域直径与最小覆盖圆半径	m	问题一
R^{rec}, U_1	有效接收半径与第一次观测后的可能源集合	m; 集合	问题二
X	第二检测点候选位置	m	问题二
$\mathcal{C}_{1000}, \mathcal{C}_{\text{det}}$	核心保证区与条件保证区	集合	问题二
J, K_{LS}	测向 Jacobian 与最小二乘误差增益矩阵	rad/m; m/rad	问题二
γ, GDOP	交会锐夹角与几何精度衰减因子	($^\circ$); m/rad	问题二
$F_1(X), f_2(X)$	最坏一阶位置误差与第二次检测耗时	m; s	问题二
$\mathcal{P}$	精度一耗时 Pareto 非支配集	集合	问题二

三、问题一：交会区域、直径与圆覆盖

3.1 测向锥与精确定位区域

定义度制方位角的单位方向向量

$$d(\phi) = \left(\cos \frac{\pi\phi}{180}, \sin \frac{\pi\phi}{180} \right)^\top.$$

在检测点 $S_i = (x_i, y_i)^\top$ 得到示向度 $\hat{\theta}_i$ 后，真实方向位于 $[\hat{\theta}_i - \delta, \hat{\theta}_i + \delta]$ 。令 $d_i^\pm = d(\hat{\theta}_i \pm \delta)$ ，二维叉积记为 $[a, b] = a_x b_y - a_y b_x$ ，则闭测向锥为

$$W_i = \{G : [d_i^-, G - S_i] \geq 0, [d_i^+, G - S_i] \leq 0\}.$$

将叉积展开，可直接得到供半平面交使用的标准形式

$$\begin{cases} \sin(\hat{\theta}_i - \delta)x - \cos(\hat{\theta}_i - \delta)y \leq \sin(\hat{\theta}_i - \delta)x_i - \cos(\hat{\theta}_i - \delta)y_i, \\ -\sin(\hat{\theta}_i + \delta)x + \cos(\hat{\theta}_i + \delta)y \leq -\sin(\hat{\theta}_i + \delta)x_i + \cos(\hat{\theta}_i + \delta)y_i. \end{cases}$$

正弦、余弦均按度制角换算为弧度后计算，故跨越 0° 不需另设分支。题目所称交会定位区域按测向锥与目标圆域定义为

$$K = B_{R_0} \cap \bigcap_{i=1}^m W_i, \quad B_{R_0} = \{G : \|G\|_2 \leq R_0\}.$$

K 是有界闭凸集；空集表示观测与先验不相容，单点和线段按退化凸集处理。正常示向度还附带 $5 < \|G - S_i\| \leq 1500 \text{ m}$ ，本问数值算例中这些距离约束不活跃；若将其作为定位区域的一部分，则外接圆盘仍保持凸性，而删除 5 m 近距盘会产生非凸开集，应按问题二的闭包口径单独处理。

3.2 半平面交与直径算法

3.2.1 圆域的双侧包围

圆弧边界不宜直接当作多边形顶点。取 $q = 720$ ，以圆周等分点的凸包构成内接正多边形 B_q^- ，以切向半平面

$$B_q^+ = \bigcap_{k=0}^{q-1} \{G : (\cos \phi_k, \sin \phi_k)G \leq R_0\}, \quad \phi_k = \frac{2k\pi}{q}$$

构成外接正多边形，并令

$$K_q^- = B_q^- \cap \bigcap_i W_i, \quad K_q^+ = B_q^+ \cap \bigcap_i W_i.$$

于是

$$K_q^- \subseteq K \subseteq K_q^+, \quad D(K_q^-) \leq D(K) \leq D(K_q^+).$$

外接多边形的最大径向超差为

$$\eta_q = R_0 \left(\sec \frac{\pi}{q} - 1 \right), \quad \eta_{720} < 0.02 \text{ m}.$$

若内包为空而外包非空，只能判定可行性未决；圆弧短到未包含内接顶点时，单纯加密不能替代连续边界证明。本文同时报告上下界；当能证明 $K_q^+ \subseteq B_{R_0}$ 时，有 $K = K_q^+$ ，外包结果可提升为精确结论。

3.2.2 逐半平面裁剪

对当前凸多边形边 XY 和单位化约束 $a^\top Z \leq r$ ，记残差 $g_X = a^\top X - r$ 、 $g_Y = a^\top Y - r$ 。若两端分居边界两侧，则交点为

$$Z = X - \frac{g_X}{g_Y - g_X}(Y - X).$$

按 Sutherland–Hodgman 规则逐边保留内侧点并插入交点，依次施加全部 $2m$ 个测向半平面 [?]。初始多边形有 q 个顶点，第 k 次裁剪后顶点数至多为 $q + k$ ，故时间复杂度为

$$O\left(\sum_{k=1}^{2m} (q + k)\right) = O(mq + m^2),$$

空间复杂度为 $O(q + m)$ 。实现中先单位化法向量，并以多边形局部中心计算残差，使容差具有长度量纲且避免大坐标消减；裁剪为空时返回不可行状态，而不是静默返回直径零。

3.2.3 凸包与旋转卡壳

凸函数 $X \mapsto \|X - Y\|_2$ 在紧凸集上的最大值可在顶点取得，故凸多边形直径必由一对凸包顶点实现。先用 Andrew 单调链去重并生成逆时针凸包 $V_0, \dots, V_{h-1}$ ，再执行下列旋转卡壳步骤 [?]：

1. 对每条边 $V_i V_{i+1}$ ，保持指针 j 指向使 $\|V_{i+1} - V_i, V_j - V_i\|$ 最大的对踵点；

2. 只要 $j+1$ 对应面积继续增大便前移 j ，并更新 (V_i, V_j) 、 (V_{i+1}, V_j) 的平方距离；
3. 面积相等时同时检查 j 与 $j+1$ ，防止平行边造成并列最远点对遗漏；扫描全部边后对最大平方距离开方。

对踵点随边界单调前进，卡壳阶段为 $O(h)$ 时间、 $O(1)$ 附加空间；含凸包构造时总复杂度为 $O(n \log n)$ 时间、 $O(n)$ 空间，输入已是严格凸边界时降为 $O(n)$ 。小规模多边形另以 $O(n^2)$ 全点对枚举校验。

3.3 直径圆的覆盖性

设 A, B 是紧集 K 的一组直径端点， $C_d = (A+B)/2$ 、 $R_d = D(K)/2$ 。恒等式

$$\|X - C_d\|_2^2 - R_d^2 = (X - A) \cdot (X - B)$$

说明点 X 位于直径圆内当且仅当右端不大于零。对凸多边形，只需检查所有顶点；边界含圆弧时，还须检查弧端点以及圆弧上背向 C_d 的驻点。

定理 1 (直径圆覆盖的充要条件)：非空紧集 K 的直径圆覆盖 K ，当且仅当

$$R_{\text{MEC}}(K) = \frac{D(K)}{2}.$$

任意覆盖圆必须包含距离为 $D(K)$ 的两点，故半径至少为 $D(K)/2$ ；若直径圆覆盖便取得等号。反之，若最小覆盖圆半径为 $D(K)/2$ ，任一对直径端点都只能是该圆的对径点，其圆心唯一为 $(A+B)/2$ ，故相应直径圆就是最小覆盖圆。

该等式并非恒成立。取边长 $D_0 = 20 \text{ m}$ 的等边三角形 $T = \text{conv}\{A, B, C\}$ ，以边 AB 为直径的圆半径为 $D_0/2$ ，而第三点到边中点的距离为 $\sqrt{3}D_0/2 > D_0/2$ 。为证明反例与题设相容，沿三条有向边的反向各延长 $L = 800 \text{ m}$ 设置检测点，并将示向锥的一条边界与相应支撑边重合、标称方向向内旋转 1° 。每个检测点所见三角形的最大角跨度为

$$\alpha = \frac{180}{\pi} \arctan \frac{10\sqrt{3}}{810} \approx 1.23^\circ < 2^\circ,$$

故三角形完整落入三个测向锥，而三个支撑半平面的交集恰为 T ；任一站到 T 的距离不超过 $L + D_0 = 820 \text{ m} < 1000 \text{ m}$ 。把三角形外心平移到原点仍满足 $R_0 = 1800 \text{ m}$ ，因此这是严格的题内反例。Jung 定理进一步给出平面有界集的普适上界

$$R_{\text{MEC}}(K) \leq \frac{D(K)}{\sqrt{3}},$$

等边三角形取等号；因 $1/\sqrt{3} > 1/2$ ，仅知直径不能保证半径 $D/2$ 的圆覆盖 [?]。

3.4 数值求解与验证

取名义源 $G_0 = (500, 300) \text{ m}$ ，三个检测点为

$$S_1 = (-300, -100), \quad S_2 = (900, -500), \quad S_3 = (-100, 1000) \quad (\text{m}),$$

对应名义示向度为 $26.57^\circ, 116.57^\circ, 310.60^\circ$ ，站源距离分别为 $894.43, 894.43, 921.95 \text{ m}$ 。三者均小于接收半径下限，且 $\|G_0\| = 583.10 \text{ m} < R_0$ 。

外包多边形所有顶点均在目标圆域内；由凸性知 $K_q^+ \subseteq B_{R_0}$ ，结合 $K \subseteq K_q^+$ 得 $K = K_q^+$ ，故表中直径是本例精确区域的结果，不依赖圆域离散误差。旋转卡壳与全部 15 个顶点对枚举一致；将顶点回代约束得到表中残差。逐顶点积判据均不大于零，独立最小覆盖圆也满足 $R_{\text{MEC}} = D/2$ ，所以本例直径圆恰能覆盖，但一般反例仍然成立。误差半角由 0.5° 增至 $1.0^\circ, 1.5^\circ$ 时，直径依次约为 $22.08, 44.14, 66.20 \text{ m}$ ，说明本构型在小误差范围内近似线性放大。

图 ?? 为便于辨认按边长 100 m 绘制相似构型；严格反例及其角跨度、接收距离证明仍取 $D_0 = 20 \text{ m}$ 、 $L = 800 \text{ m}$ ，覆盖失败不受相似缩放影响。

表 3 问题一数值结果及独立校验

指标	结果
定位区域顶点数、面积	6, 940.41 m ²
直径端点（显示值）	(520.82, 292.65), (478.94, 306.61) m
旋转卡壳直径	44.1447595094 m
最小覆盖圆半径与 $D/2$	均为 22.0723797547 m
最大半平面违反量	7.82×10^{-14} m
外包顶点最大原点距离	599.01 m < R_0

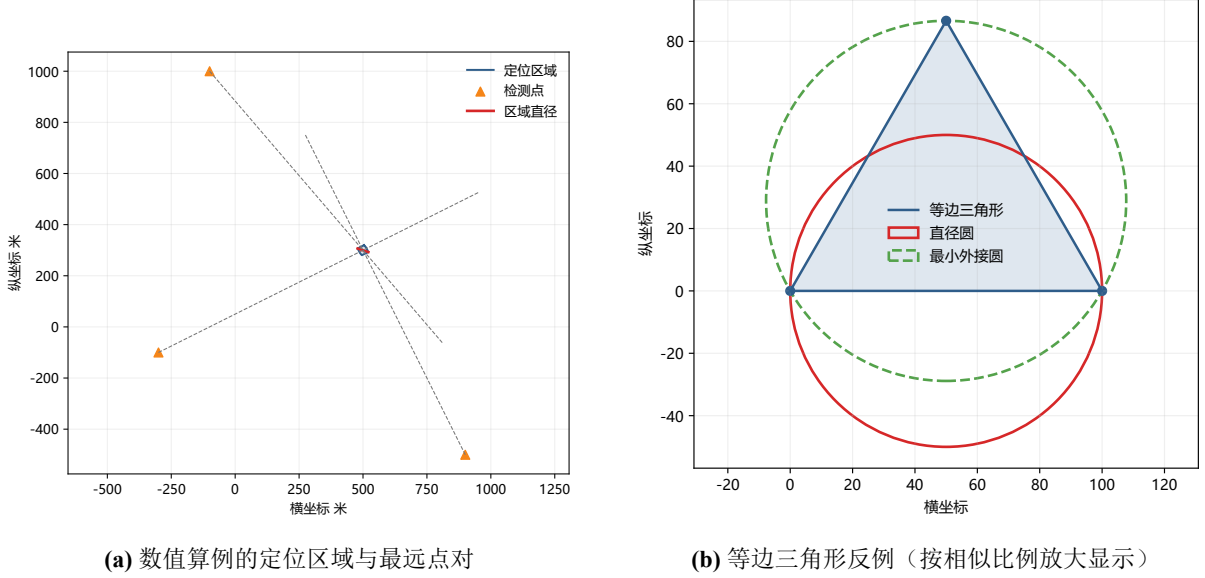


图 1 问题一的实例计算与一般性反例

四、问题二：第二检测点的稳健选址

4.1 首次观测后的不确定集合

设第一检测点为 S_1 ，正常示向度为 $\hat{\theta}$ 。真实源不仅有角度不确定性，还受正常检测的距离语义和目标圆域限制，因此完整可能源集合是二维环形扇区

$$U_1 = \left\{ G = S_1 + \rho d(\phi) : 5 < \rho \leq 1500, \left| \text{wrap}_{180}(\phi - \hat{\theta}) \right| \leq \delta, \|G\|_2 \leq R_0 \right\},$$

其中 wrap_{180} 把角差化到 $[-180^\circ, 180^\circ]$ 。把目标预先固定在名义射线或某一距离上，会同时削弱接收保证与精度评价，故不采用这种简化。

为处理一般的圆域截断，令

$$\beta(\phi) = S_1^T d(\phi), \quad \Delta(\phi) = \beta(\phi)^2 + R_0^2 - \|S_1\|_2^2.$$

当 $\Delta(\phi) \geq 0$ 时，射线落在圆域内的径向区间为

$$[\rho_-(\phi), \rho_+(\phi)] = \left[-\beta(\phi) - \sqrt{\Delta(\phi)}, -\beta(\phi) + \sqrt{\Delta(\phi)} \right],$$

故可行径向集合为 $R_f(\phi) = (5, 1500] \cap [\rho_-(\phi), \rho_+(\phi)]$ ；判别式为负或交集为空的方向直接剔除。严格下端点在极值计算中按上确界处理，避免把空集或切点退化误当作普通二维区域。

4.2 保证接收候选区及其边界

4.2.1 两层确定性保证

未知接收半径可能低至 1000 m，因此最保守的核心区为

$$\mathcal{C}_{1000} = \left\{ X : \sup_{G \in U_1} \|X - G\|_2 \leq 1000 \right\}.$$

首次检测已在 S_1 收到信号。若可能源到首站的距离为 ρ ，则该源的实际接收半径至少为 $\max(1000, \rho)$ ，由此得到更大的条件保证区

$$\mathcal{C}_{\text{det}} = \{X : \|X - G\|_2 \leq \max(1000, \|G - S_1\|_2), \forall G \in U_1\}.$$

对每个 $G \in U_1$ ，右端均不超过其实际接收半径，故 $X \in \mathcal{C}_{\text{det}}$ 仍保证第二次收到信号；并且显然有 $\mathcal{C}_{1000} \subseteq \mathcal{C}_{\text{det}}$ 。只按 1500 m 上限构造的覆盖区或面积覆盖率不能提供这一保证，故不进入主决策模型。

4.2.2 连续边界方程

取 $d_{\parallel} = d(\hat{\theta})$ 、 $d_{\perp} = (-d_{\parallel, y}, d_{\parallel, x})$ ，在第一示向局部坐标系中写成

$$X = S_1 + ad_{\parallel} + bd_{\perp}, \quad G = S_1 + \rho(\cos \eta d_{\parallel} + \sin \eta d_{\perp}), \quad |\eta| \leq \delta.$$

于是正确的局部距离平方为

$$L^2(a, b; \rho, \eta) = (a - \rho \cos \eta)^2 + (b - \rho \sin \eta)^2. \quad (1)$$

式 (1) 只含相对角 η ；若在该局部坐标中仍写 $\hat{\theta} + \eta$ ，便会重复旋转坐标轴。核心区的连续残差为

$$F_{\text{core}}(a, b) = \sup_{\substack{|\eta| \leq \delta \\ \rho \in R_f(\hat{\theta} + \eta)}} L(a, b; \rho, \eta) - 1000,$$

从而

$$\mathcal{C}_{1000} = \{S_1 + ad_{\parallel} + bd_{\perp} : F_{\text{core}}(a, b) \leq 0\}, \quad \partial \mathcal{C}_{1000} : F_{\text{core}}(a, b) = 0. \quad (2)$$

当目标圆域不截断 $5 < \rho \leq 1500$ 的扇形时， $\mathcal{C}_{\text{det}}$ 可进一步化简。对 $\rho > 1000$ ，令同向辅助点 $G_0 = S_1 + 1000d(\phi)$ ；若 $\|X - G_0\| \leq 1000$ ，则

$$\|X - G\| \leq \|X - G_0\| + (\rho - 1000) \leq \rho.$$

因此完整条件与只检查 $5 < \rho \leq 1000$ 等价，边界可写为

$$F_{\text{det}}(a, b) = \sup_{\substack{|\eta| \leq \delta \\ 5 < \rho \leq 1000}} L(a, b; \rho, \eta) - 1000 = 0. \quad (3)$$

固定 η 时， L^2 关于 ρ 为凸二次函数，最大值只在径向端点或开端极限取得；固定端点时，角度候选为 $\eta = \pm\delta$ 及满足

$$a \sin \eta - b \cos \eta = 0$$

的区间内驻点。故式 (2)、(3) 都是有限支函数的上包络，可非采样地判断边界内外。一般圆域截断时，式 (2) 仍是完整核心区；程序对 $\mathcal{C}_{\text{det}}$ 使用未截断楔形得到安全内近似，不把它声称完整边界。下文算例满足 $S_1 = (0, 0)$ 且 $1500 < R_0$ ，不存在截断，故式 (3) 给出的条件保证区是精确的。

4.3 精度—耗时 Pareto 模型

4.3.1 局部耗时

机器狗从 S_1 直线移动到 X 并完成第二次测向的耗时为

$$f_2(X) = \frac{\|X - S_1\|_2}{5} + 5 + \mathbf{1}_{\text{switch}}.$$

连续测同一频道时末项为零；若中间插入其他频道动作则为 1。设局部预算为 $T_{\max}$ ，以 $\mathcal{C}_{\det}(T_{\max}) = \{X \in \mathcal{C}_{\det} : f_2(X) \leq T_{\max}\}$ 作为主可行区。 $T_{\max}$ 是可调决策参数，不是题面常量。

4.3.2 误差传播与 GDOP

设 $G = (x, y)^\top$ ，第 i 个检测点为 (x_i, y_i) ， $r_i = \|G - S_i\|_2$ 。弧度制测向函数的 Jacobian 行向量为

$$J_i = \begin{bmatrix} -(y - y_i)/r_i^2 & (x - x_i)/r_i^2 \end{bmatrix}, \quad J = \begin{bmatrix} J_1 \\ J_2 \end{bmatrix}.$$

当 J 满秩时，最小二乘误差增益矩阵为 $K_{\text{LS}} = (J^\top J)^{-1} J^\top$ 。令 $\tilde{\delta} = \pi\delta/180$ ，角误差盒的四个顶点为 $u \in \{-1, 1\}^2$ ，则固定源位置的一阶最坏位置误差为

$$f_{1,\text{loc}}(G; X) = \tilde{\delta} \max_{u \in \{-1, 1\}^2} \|K_{\text{LS}} u\|_2, \quad F_1(X) = \sup_{G \in U_1} f_{1,\text{loc}}(G; X). \quad (4)$$

线性映射把误差盒变为平行四边形；若两视线锐夹角为 γ ，其面积为

$$A_{\text{loc}} = \frac{4\tilde{\delta}^2 r_1 r_2}{|\sin \gamma|} \leq \pi f_{1,\text{loc}}^2 \leq \pi F_1(X)^2.$$

据此取 $A_{\text{proxy}}(X) = \pi F_1(X)^2$ 作为面积代理。它是局部线性化误差集合包含圆面积的上界，不是两次读数后真实定位区域的面积，也不是其最小覆盖圆半径。作为独立几何诊断量，定义

$$\text{GDOP} = \sqrt{\text{tr}[(J^\top J)^{-1}]} = \frac{\sqrt{r_1^2 + r_2^2}}{|\sin \gamma|} \quad (\text{m/rad}),$$

它揭示“距离短、交角近正交”共同改善精度，而不引入误差分布 [?]。

4.3.3 双目标决策

连续先验在计算中离散为 $U_{1,N}$ ，记 $\hat{F}_{1,N} = \max_{G \in U_{1,N}} f_{1,\text{loc}}(G; X)$ 。在保证接收、非近距和非共线条件下求

$$\min_X (\hat{F}_{1,N}(X), f_2(X)), \quad \text{s.t. } X \in \mathcal{C}_{\det}(T_{\max}), \quad \inf_{G \in U_{1,N}} \|X - G\| > 5, \quad \inf_{G \in U_{1,N}} |\sin \gamma| \geq 10^{-12}. \quad (5)$$

若不存在另一可行点在两项目标上均不劣且至少一项严格更优，则 X 属于 Pareto 集 $\mathcal{P}$ 。本文先保留 $\mathcal{P}$ ，再按“最小化 $\hat{F}_{1,N}$ ，相同时最小化 f_2 ”作字典序选择，避免把米与秒用任意权重相加。有限样本只负责搜索；对最终站址再在连续 U_1 上作区间认证。

4.4 第二次观测后的清除证书

第二次正常示向度返回后，不能用式 (4) 的代理直接决定清除。应重新构造

$$P_{\text{exact}}^{(2)} = \{G : \|G\| \leq R_0, G \in W_1 \cap W_2, 5 < \|G - S_i\| \leq 1500 \ (i = 1, 2)\}.$$

其闭凸外包由目标圆、两个测向锥和两个 1500 m 圆盘相交得到，再显式删除两个 5 m 近距盘；直径按上确界、最小覆盖圆按闭包计算。存在同一清除点覆盖所有可能位置的充要条件为

$$R_{\text{MEC}}(\overline{P_{\text{exact}}^{(2)}}) \leq 20 \text{ m}. \quad (6)$$

若第二次检测返回“距离过近”，则直接进入近距清除分支；若区域为空，则报告几何不相容而非零误差。

4.5 算例与结果分析

取 $S_1 = (0, 0)$ 、 $\hat{\theta} = 25^\circ$ 、 $\delta = 1^\circ$ 、 $T_{\max} = 260$ s。此时 $U_1 = \{\rho d(25^\circ + \eta) : 5 < \rho \leq 1500, |\eta| \leq 1^\circ\}$ ，圆域不截断扇形。在局部坐标网格 $a = 0, 10, \dots, 1500$ 、 $b = -800, -790, \dots, 800$ 上，条件保证区保留 12093 点，保守核心区保留 4464 点，Pareto 前沿含 1154 点。字典序选得

$$S_2 = (993.738585, -143.469943) \text{ m} = S_1 + 840d_{\parallel} - 550d_{\perp}.$$

表 4 问题二参考站址的数值结果

指标	结果
第二次检测局部耗时 $f_2(S_2)$	205.808366 s
完整先验上的最大站源距离	999.911130 m
连续域 $F_1(S_2)$ 认证区间	$[60.8760257892, 60.8769672571]$ m
加密样本下最坏 GDOP	2707.663746 m/rad
有限样本面积代理 $\hat{A}_N$	$1.16 \times 10^4 \text{ m}^2$
连续先验的最小站源距离	535.26 m > 5 m
连续先验的最小交会正弦下界	> 0.53

对推荐点采用 85 个区间分支包围连续先验，认证区间宽度小于 10^{-3} m；最大站源距离则由式 (1) 的端点—驻点枚举直接取得，故“保证接收”不是有限采样推断。图 ?? 给出条件保证候选区、精度—耗时非支配前沿与参考站址。

参考站址方向与首次示向锥的最小线夹角保守取 $\gamma_0 \geq 32.21^\circ$ 。利用 $[G - S_1, X - G] = [G - S_1, X - S_1]$ 、 $\|X - S_1\| > 1004.04$ m 以及 $\|X - G\| \leq 999.92$ m，对全部 $G \in U_1$ 有

$$|\sin \gamma| = \frac{\|X - S_1\| \sin \gamma_0}{\|X - G\|} \geq \frac{1004.04}{999.92} \sin 32.21^\circ > 0.53,$$

从而表 ?? 中的非退化下界也建立在连续先验上，而非有限源点样本上。

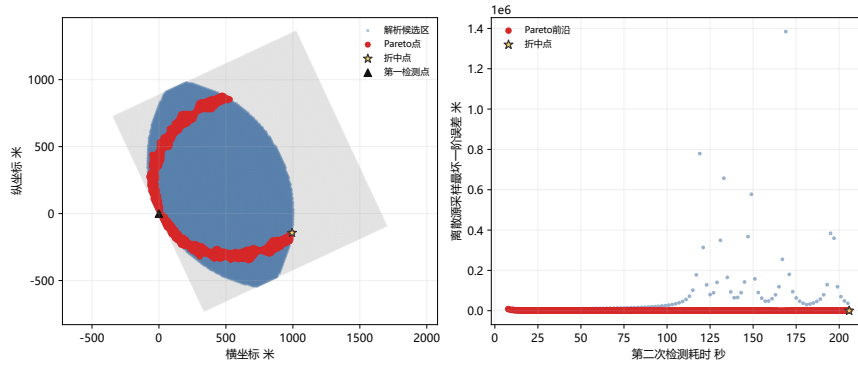


图 2 第二检测点条件保证候选区、Pareto 前沿与参考站址（程序图中的“解析候选区”“折中点”分别对应本文的条件保证区与参考站址）

参考点同时属于 C_{1000} ，但距 1000 m 边界仅约 0.09 m，部署时须保留额外模型余量。这里的“最优”只指给定搜索网格和字典序规则下的候选最优；区间算法认证的是该站址的连续先验最坏误差，而非连续站址域的全局最优。又因 $F_1 \approx 60.88$ m 与实际区域的 MEC 半径不是同一量，是否能够清除仍必须等待第二次示向度并检验式 (6)。

五、问题三的模型建立与求解

5.1 模型准备

5.2 模型建立

5.3 模型求解与结果分析

5.4 小结

六、问题四的模型建立与求解

6.1 模型准备

6.2 模型建立

6.3 模型求解与结果分析

6.4 小结

七、模型的检验

7.1 灵敏度分析

7.2 模型检验与误差分析

7.3 稳健性分析

八、模型评价、改进与推广

8.1 模型评价

主要优点如下：

1. **保证口径完整。**问题一、二始终在 $\pm 1^\circ$ 误差盒和未知接收半径上取确定性最坏情形；问题三用距离覆盖认证全向源不存在，问题四用连续域凸包条件消除定向盲区三义性，均不以有限随机采样替代完备证书。
2. **几何结论可核验。**问题一同时给出圆域内外包、半平面残差、卡壳—暴力一致性和严格题内反例；问题二区分核心保证区与条件保证区，并对推荐站址的连续最坏误差给出宽度小于 10^{-3} m 的区间认证。
3. **精度与效率分层。**一阶误差传播和 GDOP 只用于观测前排序，观测后仍以实际可行集的最小覆盖圆作清除证书，避免把代理误当作物理阈值。在线算法则把扫描、顺路观测和清除节点统一规划，减少无效往返。
4. **实测闭合。**冻结版本的三次正式测试中，问题三清除 40 个源、空清除 0 次，平均每源耗时 289.91 s；问题四清除 39 个源、空清除 2 次，平均每源耗时 462.66 s。各局 20 个频道均取得“已清除”或“确认不存在”证书，程序运行时间均小于 2.5 s。

局限性如下：

1. 题面未给示向误差分布，确定性模型不能利用概率平均效应；其代价是结果偏保守，但换取了无需额外统计假设的最坏情形保证。

2. 问题二的推荐点是给定网格和字典序规则下的候选最优，连续认证只证明该点的指标界，不证明连续站址域的全局最优；其最大接收距离距边界约 0.09 m，对题面外的传播扰动较敏感。
3. 六次正式测试足以验证接口闭合和案例内正确性，却不足以精确估计随机案例总体的均值与尾部风险；效率结论还需结合多局演练分布解释。

8.2 改进与推广

在不改变确定性证书的前提下，可用区间分支与支撑函数补齐一般圆域截断下的完整条件保证区，并以连续优化替代问题二的固定网格；在线部分可继续把认证站、观测站和清除点统一为带证据约束的开放路径问题，采用滚动时域与局部精确路由压缩移动常数项。测向锥交、连续覆盖证书和证据约束路由也可迁移到无人机搜救、移动传感器部署及其他无源定位任务，只需替换观测集合与动作代价。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，用于备选建模思路比较、程序审查与辅助实现、测试用例生成、语言润色及排版。参赛队对题意解读、模型选择、程序结果和提交内容进行了人工复核，并对最终作品负责。详细使用情况见支撑材料。

参考文献

- [1] 王凯. 无线电测向定位方法在新型无线电干扰查找中的运用研究[D]. [出版地不详]: 南昌大学, 2019.
- [2] SUTHERLAND I E, HODGMAN G W. Reentrant polygon clipping[J/OL]. Communications of the ACM, 1974, 17(1):32-42. DOI: 10.1145/360767.360802.
- [3] TOUSSAINT G T. Applications of the rotating calipers to geometric problems in two and three dimensions[J/OL]. International Journal of Digital Information and Wireless Communications, 2014, 4(3): 372-386. DOI: 10.17781/P001290.
- [4] JUNG H. Ueber die kleinste kugel, die eine räumliche figur einschliesst[J/OL]. Journal für die reine und angewandte Mathematik, 1901, 1901(123):241-257. DOI: 10.1515/crll.1901.123.241.
- [5] BISHOP A N, FIDAN B, ANDERSON B D O, et al. Optimality analysis of sensor-target localization geometries[J/OL]. Automatica, 2010, 46(3):479-492. DOI: 10.1016/j.automatica.2009.12.003.

附录 A 支撑材料文件清单

本文支撑材料按用途分为三部分，目录结构如下：

支撑材料/

├— 行为日志/	六份正式测试行为日志及其元数据
├— 程序代码/	robot 程序包与几何库
└— 结果数据/	正式与演练动作日志、各问数值结果

附录 B 程序代码

本附录列出各问的核心程序。编号 1–12 是问题一、二使用的几何库；编号 13–22a 是问题三、四未提速的基础实现，用于展示发现证书与状态机主干；编号 23–29 是六次正式测试所用提交版本的路径与测站优化。代码中不含注释与文档字符串，逻辑与支撑材料 程序代码/ 中的源码一致；每段代码左侧的行号与原文件一致。

B.1 问题一代码

编号 1 单次示向度化为两个半平面 (bearing_halfplanes, 对应论文三 §1.1)

```
78 def bearing_halfplanes(site: Sequence[float], bearing_deg: float,
79                        err_deg: float = 1.0) -> Arr:
80     if (not math.isfinite(bearing_deg) or not math.isfinite(err_deg)
81         or not 0 <= err_deg < 90):
82         raise ValueError("err_deg must be in [0, 90)")
83     s = _point(site, "site")
84     lo = math.radians(bearing_deg - err_deg)
85     hi = math.radians(bearing_deg + err_deg)
86     d1 = np.array([math.cos(lo), math.sin(lo)])
87     dh = np.array([math.cos(hi), math.sin(hi)])
88     h1 = np.array([d1[1], -d1[0], d1[1] * s[0] - d1[0] * s[1]])
89     h2 = np.array([-dh[1], dh[0], -dh[1] * s[0] + dh[0] * s[1]])
90     return np.vstack((h1, h2))
```

编号 2 内外接多边形给出直径上下界 (localization_diameter_bounds, 对应论文三 §1.2)

```
184 def localization_diameter_bounds(
185     sites: Sequence[Sequence[float]] | Arr,
186     bearings_deg: Sequence[float] | Arr,
187     err_deg: float = 1.0, rad: float = 1800.0,
188     n_bound: int = 720,
189     center: Sequence[float] = (0.0, 0.0),
190 ) -> dict[str, Arr | float]:
191     outer_poly, hps = localization_polygon(
192         sites, bearings_deg, err_deg, rad, n_bound, center, outer=True)
193     if len(outer_poly) == 0:
194         raise ValueError("bearing constraints have empty intersection")
195     inner_poly, _ = localization_polygon(
196         sites, bearings_deg, err_deg, rad, n_bound, center, outer=False)
```

```

197 upper, upper_pair = polygon_diameter(outer_poly)
198 if len(inner_poly):
199     lower, lower_pair = polygon_diameter(inner_poly)
200     if lower > upper:
201         slack = eps * max(lower, upper, np.finfo(float).tiny)
202         if lower - upper > slack:
203             raise RuntimeError("inner diameter exceeds outer diameter")
204         lower = upper
205     else:
206         lower = 0.0
207         lower_pair = np.empty((0, 2), dtype=float)
208 return {
209     "inner_poly": inner_poly,
210     "outer_poly": outer_poly,
211     "halfplanes": hps,
212     "lower_m": lower,
213     "upper_m": upper,
214     "lower_pair": lower_pair,
215     "upper_pair": upper_pair,
216 }

```

编号3 逐半平面裁剪凸多边形 (clip_halfplane, 对应论文三 §2.1)

```

106 def clip_halfplane(poly: Sequence[Sequence[float]] | Arr,
107                    hp: Sequence[float] | Arr, tol: float = eps) -> Arr:
108     p = _pts(poly)
109     tol = _relative_tol(tol)
110     h = np.asarray(hp, dtype=float)
111     norm = float(np.linalg.norm(h[:2])) if h.shape == (3,) else 0.0
112     if (h.shape != (3,) or not np.all(np.isfinite(h))
113         or norm <= np.finfo(float).tiny):
114         raise ValueError("half-plane must be [a, b, c] with nonzero normal")
115     if len(p) == 0:
116         return p
117     org = p.mean(axis=0)
118     nrm = h[:2] / norm
119     cut = float((h[2] - h[:2] @ org) / norm)
120     q = p - org
121     span = max(float(np.ptp(q[:, 0])), float(np.ptp(q[:, 1])),
122                np.finfo(float).tiny)
123     roundoff = 32.0 * np.finfo(float).eps * max(
124         float(np.linalg.norm(org)), abs(cut), span)
125     cut_tol = tol * span + roundoff
126     out: list[Arr] = []
127     for i, a in enumerate(q):
128         b = q[(i + 1) % len(q)]
129         fa = float(nrm @ a - cut)
130         fb = float(nrm @ b - cut)

```

```

131     ina, inb = fa <= cut_tol, fb <= cut_tol
132     if ina:
133         out.append(a)
134     if ina != inb:
135         den = fa - fb
136         den_tol = 32.0 * np.finfo(float).eps * max(
137             abs(fa), abs(fb), span)
138         if abs(den) > den_tol:
139             out.append(a + fa / den * (b - a))
140     if not out:
141         return np.empty((0, 2), dtype=float)
142     return _clean_poly(np.asarray(out, dtype=float) + org, tol)

```

编号4半平面交主循环与定位区域组装(halfplane_intersection / localization_polygon, 对应论文三 §2.1)

```

144 def halfplane_intersection(hps: Sequence[Sequence[float]] | Arr,
145                           bound: Sequence[Sequence[float]] | Arr,
146                           tol: float = eps) -> Arr:
147     h = np.asarray(hps, dtype=float)
148     tol = _relative_tol(tol)
149     if h.size == 0:
150         return _pts(bound).copy()
151     if h.ndim != 2 or h.shape[1] != 3 or not np.all(np.isfinite(h)):
152         raise ValueError("hps must have shape (n, 3)")
153     p = _pts(bound).copy()
154     for row in h:
155         p = clip_halfplane(p, row, tol)
156         if len(p) == 0:
157             break
158     return p

```

```

160 def localization_polygon(sites: Sequence[Sequence[float]] | Arr,
161                         bearings_deg: Sequence[float] | Arr,
162                         err_deg: float = 1.0, rad: float = 1800.0,
163                         n_bound: int = 720,
164                         center: Sequence[float] = (0.0, 0.0),
165                         outer: bool = True) -> tuple[Arr, Arr]:
166     s = _pts(sites)
167     ang = np.asarray(bearings_deg, dtype=float)
168     if ang.shape != (len(s),) or not np.all(np.isfinite(ang)):
169         raise ValueError("one bearing is required for each site")
170     if len(s) == 0:
171         raise ValueError("at least one bearing is required")
172     c = _point(center, "center")
173     local_sites = s - c
174     local_hps = np.vstack([
175         bearing_halfplanes(p, a, err_deg) for p, a in zip(local_sites, ang)

```

```

176     ])
177     local_poly = halfplane_intersection(
178         local_hps, regular_bound(rad, n_bound, outer=outer))
179     global_hps = np.vstack([
180         bearing_halfplanes(p, a, err_deg) for p, a in zip(s, ang)
181     ])
182     return local_poly + c, global_hps

```

编号5 凸包加旋转卡壳求直径与最远点对 (polygon_diameter, 对应论文三 §2.2)

```

218 def polygon_diameter(x: Sequence[Sequence[float]] | Arr,
219                     tol: float = eps) -> tuple[float, Arr]:
220     raw = _pts(x)
221     tol = _relative_tol(tol)
222     if len(raw) == 0:
223         raise ValueError("diameter is undefined for an empty set")
224     org = raw.mean(axis=0)
225     p = convex_hull(raw - org, tol)
226     n = len(p)
227     if n == 1:
228         return 0.0, np.vstack((p[0], p[0])) + org
229     if n == 2:
230         return float(np.linalg.norm(p[1] - p[0])), p.copy() + org
231     span = max(float(np.ptp(p[:, 0])), float(np.ptp(p[:, 1])),
232               np.finfo(float).tiny)
233     area_tol = tol * span * span
234     dist2_tol = tol * span * span
235     j = 1
236     best = -1.0
237     pair = np.vstack((p[0], p[1]))
238
239     def upd(a: int, b: int) -> None:
240         nonlocal best, pair
241         val = float((p[a] - p[b]) @ (p[a] - p[b]))
242         if val > best + dist2_tol:
243             best = val
244             pair = np.vstack((p[a], p[b]))
245
246     for i in range(n):
247         ni = (i + 1) % n
248         while True:
249             nj = (j + 1) % n
250             cur = abs(cross(p[ni] - p[i], p[j] - p[i]))
251             nxt = abs(cross(p[ni] - p[i], p[nj] - p[i]))
252             if nxt > cur + area_tol:
253                 j = nj
254             else:
255                 break

```

```

256     upd(i, j)
257     upd(ni, j)
258     nj = (j + 1) % n
259     cur = abs(cross(p[ni] - p[i], p[j] - p[i]))
260     nxt = abs(cross(p[ni] - p[i], p[nj] - p[i]))
261     if abs(nxt - cur) <= area_tol:
262         upd(i, nj)
263         upd(ni, nj)
264     return math.sqrt(max(best, 0.0)), pair + org

```

B.2 问题二代码

编号 6 方位 Jacobian 与加权最小二乘误差传播 (bearing_jacobian / error_propagation, 对应论文四 §1)

```

336 def bearing_jacobian(target: Sequence[float],
337                     sites: Sequence[Sequence[float]] | Arr) -> Arr:
338     g = _point(target, "target")
339     s = _pts(sites)
340     if len(s) == 0:
341         raise ValueError("at least one site is required")
342     d = g - s
343     r2 = np.sum(d * d, axis=1)
344     if np.any(r2 <= np.finfo(float).tiny):
345         raise ValueError("target and site must be distinct")
346     return np.column_stack((-d[:, 1] / r2, d[:, 0] / r2))

```

```

359 def error_propagation(target: Sequence[float],
360                     sites: Sequence[Sequence[float]] | Arr,
361                     sig_deg: float | Sequence[float] = 1.0 / math.sqrt
362                     (3.0),
363                     tol: float = rank_rel_tol) -> dict[str, Arr |
364                     float]:
365     tol = _relative_tol(tol)
366     h = bearing_jacobian(target, sites)
367     sig = np.asarray(sig_deg, dtype=float)
368     if sig.ndim == 0:
369         sig = np.full(len(h), float(sig))
370     if (sig.shape != (len(h),) or not np.all(np.isfinite(sig))
371         or np.any(sig <= 0)):
372         raise ValueError("sig_deg must be positive for each site")
373     sig = np.radians(sig)
374     r = np.diag(sig * sig)
375     w = np.diag(1.0 / (sig * sig))
376     info = h.T @ w @ h
377     val = np.linalg.eigvalsh(info)
378     if val[-1] <= 0.0 or val[0] <= tol * val[-1]:

```

```

377     raise ValueError("bearing geometry is singular")
378 cov = np.linalg.solve(info, np.eye(2))
379 gain = np.linalg.solve(info, h.T @ w)
380 return {
381     "h": h,
382     "r": r,
383     "gain": gain,
384     "cov": cov,
385     "gdop": gdop(target, sites, tol),
386     "rms": math.sqrt(float(np.trace(cov))),
387 }

```

编号7 首次观测后的可能源集合采样 (source_cone_samples, 对应论文四 §2)

```

507 def source_cone_samples(
508     site: Sequence[float], bearing_deg: float,
509     ranges: tuple[float, float] = (5.0, 1500.0),
510     err_deg: float = 1.0, target_rad: float = 1800.0,
511     n_range: int = 41, n_angle: int = 21,
512 ) -> Arr:
513     s = _point(site, "site")
514     lo, hi = ranges
515     if (not math.isfinite(bearing_deg) or not math.isfinite(lo)
516         or not math.isfinite(hi) or not math.isfinite(err_deg)
517         or not 0 <= lo < hi or not 0 <= err_deg < 90):
518         raise ValueError("invalid cone ranges or angle error")
519     if (not math.isfinite(target_rad) or target_rad <= 0
520         or not isinstance(n_range, int) or isinstance(n_range, bool)
521         or not isinstance(n_angle, int) or isinstance(n_angle, bool)
522         or n_range < 2 or n_angle < 2):
523         raise ValueError("invalid disk radius or sample counts")
524     offsets = np.unique(np.append(
525         np.linspace(-err_deg, err_deg, n_angle), 0.0))
526     angles = np.radians(bearing_deg + offsets)
527     samples = []
528     for angle in angles:
529         direction = np.array([math.cos(angle), math.sin(angle)])
530         projection = float(s @ direction)
531         discriminant = projection * projection + target_rad * target_rad -
532         float(s @ s)
533         if discriminant < 0.0:
534             continue
535         root = math.sqrt(max(discriminant, 0.0))
536         feasible_lo = max(lo, -projection - root)
537         feasible_hi = min(hi, -projection + root)
538         if feasible_lo > feasible_hi + eps:
539             continue
540         rho = np.linspace(feasible_lo, feasible_hi, n_range)

```

```

540     samples.append(s + rho[:, None] * direction)
541 if not samples:
542     return np.empty((0, 2), dtype=float)
543 return np.vstack(samples)

```

编号 8 保证接收的解析边界 (sector_max_distance, 对应论文四 §3.2)

```

545 def sector_max_distance(
546     cand: Sequence[Sequence[float]] | Arr,
547     site: Sequence[float], bearing_deg: float,
548     ranges: tuple[float, float] = (5.0, 1500.0),
549     err_deg: float = 1.0,
550 ) -> Arr:
551     p = _pts(cand)
552     s = _point(site, "site")
553     lo, hi = ranges
554     if (not math.isfinite(bearing_deg) or not math.isfinite(lo)
555         or not math.isfinite(hi) or not math.isfinite(err_deg)
556         or not 0 <= lo <= hi or not 0 <= err_deg < 90):
557         raise ValueError("invalid cone ranges or angle error")
558     ang = math.radians(bearing_deg)
559     u = np.array([math.cos(ang), math.sin(ang)])
560     v = np.array([-math.sin(ang), math.cos(ang)])
561     d = p - s
562     a, b = d @ u, d @ v
563     err = math.radians(err_deg)
564     end_min = np.minimum(
565         a * math.cos(err) + b * math.sin(err),
566         a * math.cos(err) - b * math.sin(err),
567     )
568     phase = np.arctan2(b, a)
569     opp = (phase + math.pi + math.pi) % (2.0 * math.pi) - math.pi
570     dot_min = np.where(np.abs(opp) <= err + eps, -np.hypot(a, b), end_min)
571     q_lo = np.sum(d * d, axis=1) + lo * lo - 2.0 * lo * dot_min
572     q_hi = np.sum(d * d, axis=1) + hi * hi - 2.0 * hi * dot_min
573     return np.sqrt(np.maximum(np.maximum(q_lo, q_hi), 0.0))

```

编号 9 确定性最坏误差界与耗时 (second_site_metrics_sources, 对应论文四 §4.1)

```

626 def second_site_metrics_sources(
627     cand: Sequence[Sequence[float]] | Arr,
628     site1: Sequence[float], sources: Sequence[Sequence[float]] | Arr,
629     sig_deg: float = 1.0 / math.sqrt(3.0),
630     err_deg: float = 1.0,
631     recv_rad: float = 1000.0, quantile: float = 1.0,
632     speed: float = 5.0, measure_s: float = 5.0,
633     switched: bool = False, switch_s: float = 1.0,
634 ) -> dict[str, Arr]:
635     p = _pts(cand)

```

```

636 g = _pts(sources)
637 s = _point(site1, "site1")
638 if len(g) == 0 or not 0 <= quantile <= 1:
639     raise ValueError("sources must be nonempty and quantile valid")
640 if (not math.isfinite(sig_deg) or not math.isfinite(err_deg)
641     or sig_deg <= 0 or err_deg <= 0):
642     raise ValueError("angle scales must be finite and positive")
643 if (not math.isfinite(recv_rad) or recv_rad <= 0
644     or not math.isfinite(speed) or speed <= 0
645     or not math.isfinite(measure_s) or measure_s < 0
646     or not math.isfinite(switch_s) or switch_s < 0):
647     raise ValueError("invalid reception or time parameters")
648 first = s[None, :] - g
649 r1 = np.linalg.norm(first, axis=1)
650 second = p[:, None, :] - g[None, :, :]
651 r2 = np.linalg.norm(second, axis=2)
652 dot = np.sum(first[None, :, :] * second, axis=2)
653 den = r1[None, :] * r2
654 cos_val = np.abs(dot) / np.maximum(den, np.finfo(float).tiny)
655 cos_val = np.clip(cos_val, 0.0, 1.0)
656 cross_val = (first[None, :, 0] * second[:, :, 1]
657             - first[None, :, 1] * second[:, :, 0])
658 sin_val = np.abs(cross_val) / np.maximum(den, np.finfo(float).tiny)
659 sin_val = np.clip(sin_val, 0.0, 1.0)
660 unsafe = ((r1[None, :] <= np.finfo(float).tiny)
661          | (r2 <= np.finfo(float).tiny)
662          | (sin_val <= sin_angle_tol))
663 raw = np.sqrt(r1[None, :] ** 2 + r2 * r2) / np.where(
664     unsafe, math.nan, sin_val)
665 raw[unsafe] = math.inf
666 h1x = first[:, 1] / np.maximum(r1 * r1, np.finfo(float).tiny)
667 h1y = -first[:, 0] / np.maximum(r1 * r1, np.finfo(float).tiny)
668 h2x = second[:, :, 1] / np.maximum(r2 * r2, np.finfo(float).tiny)
669 h2y = -second[:, :, 0] / np.maximum(r2 * r2, np.finfo(float).tiny)
670 det = h1x[None, :] * h2y - h1y[None, :] * h2x
671 safe_det = np.where(unsafe, math.nan, det)
672 k1x = h2y / safe_det
673 k1y = -h2x / safe_det
674 k2x = -h1y[None, :] / safe_det
675 k2y = h1x[None, :] / safe_det
676 plus = np.hypot(k1x + k2x, k1y + k2y)
677 minus = np.hypot(k1x - k2x, k1y - k2y)
678 bound = math.radians(err_deg) * np.maximum(plus, minus)
679 bound[~np.isfinite(bound)] = math.inf
680 if quantile == 1.0:
681     ang_cost = np.max(cos_val, axis=1)
682     gdop_val = np.max(raw, axis=1)

```

```

683     sampled_error = np.max(bound, axis=1)
684 elif quantile == 0.0:
685     ang_cost = np.min(cos_val, axis=1)
686     gdop_val = np.min(raw, axis=1)
687     sampled_error = np.min(bound, axis=1)
688 else:
689     ang_cost = np.quantile(cos_val, quantile, axis=1)
690     gdop_val = np.quantile(raw, quantile, axis=1)
691     sampled_error = np.quantile(bound, quantile, axis=1)
692 rms = math.radians(sig_deg) * gdop_val
693 coverage = np.mean(r2 <= recv_rad + eps, axis=1)
694 detection_coverage = np.mean(
695     r2 <= np.maximum(recv_rad, r1)[None, :] + eps, axis=1)
696 time_s = (np.linalg.norm(p - s, axis=1) / speed + measure_s
697     + switch_s * float(swapped))
698 return {
699     "angle_cost": ang_cost,
700     "gdop": gdop_val,
701     "rms": rms,
702     "sampled_error": sampled_error,
703     "time_s": time_s,
704     "coverage": coverage,
705     "detection_coverage": detection_coverage,
706 }

```

编号 10 非支配筛选 (pareto_mask, 对应论文四 §4.1)

```

807 def pareto_mask(vals: Sequence[Sequence[float]] | Arr,
808                 tol: float = eps) -> Arr:
809     x = np.asarray(vals, dtype=float)
810     tol = _relative_tol(tol)
811     if x.ndim != 2 or len(x) == 0:
812         raise ValueError("vals must have shape (n, m) with n positive")
813     ok = np.all(np.isfinite(x), axis=1)
814     ids = np.flatnonzero(ok)
815     keep = np.zeros(len(x), dtype=bool)
816     if len(ids) == 0:
817         return keep
818     scale = np.maximum(np.max(np.abs(x[ids])), axis=0),
819                     np.finfo(float).tiny)
820     cut = tol * scale
821     for i in ids:
822         dom = np.all(x[ids] <= x[i] + cut, axis=1)
823         strict = np.any(x[ids] < x[i] - cut, axis=1)
824         if not np.any(dom & strict):
825             keep[i] = True
826     return keep

```

编号 11 保证接收区筛选 (robust_candidate_mask, 对应论文四 §4.2)

```
575 def robust_candidate_mask(  
576     cand: Sequence[Sequence[float]] | Arr,  
577     site1: Sequence[float], bearing_deg: float,  
578     ranges: tuple[float, float] = (5.0, 1500.0),  
579     err_deg: float = 1.0, recv_rad: float = 1000.0,  
580     time_limit_s: float = math.inf, speed: float = 5.0,  
581     measure_s: float = 5.0, switched: bool = False,  
582     switch_s: float = 1.0,  
583 ) -> Arr:  
584     p = _pts(cand)  
585     s = _point(site1, "site1")  
586     if (not math.isfinite(recv_rad) or recv_rad <= 0  
587         or not math.isfinite(speed) or speed <= 0  
588         or not math.isfinite(measure_s) or measure_s < 0  
589         or not math.isfinite(switch_s) or switch_s < 0  
590         or math.isnan(time_limit_s)):  
591         raise ValueError("invalid reception or time parameters")  
592     far = sector_max_distance(p, s, bearing_deg, ranges, err_deg)  
593     t = (np.linalg.norm(p - s, axis=1) / speed + measure_s  
594         + switch_s * float(switched))  
595     return (far <= recv_rad + eps) & (t <= time_limit_s + eps)
```

编号 12 双目标 Pareto 选点主流程 (pareto_second_sites_cone, 对应论文四 §4.2)

```
842 def pareto_second_sites_cone(  
843     cand: Sequence[Sequence[float]] | Arr,  
844     site1: Sequence[float], bearing_deg: float,  
845     ranges: tuple[float, float] = (5.0, 1500.0),  
846     err_deg: float = 1.0, recv_rad: float = 1000.0,  
847     quantile: float = 1.0, min_coverage: float = 1.0,  
848     n_range: int = 41, n_angle: int = 21,  
849     switched: bool = False, conditional_reception: bool = True,  
850 ) -> dict[str, Arr]:  
851     src = source_cone_samples(  
852         site1, bearing_deg, ranges, err_deg, n_range=n_range, n_angle=  
853             n_angle)  
854     res = second_site_metrics_sources(  
855         cand, site1, src, err_deg=err_deg, recv_rad=recv_rad,  
856         quantile=quantile,  
857         switched=switched)  
858     vals = np.column_stack((res["sampled_error"], res["time_s"]))  
859     if conditional_reception:  
860         guaranteed = detected_candidate_mask(  
861             cand, site1, bearing_deg, ranges, err_deg, recv_rad,  
862             switched=switched)  
863         sampled_coverage = res["detection_coverage"]
```

```

863     else:
864         guaranteed = robust_candidate_mask(
865             cand, site1, bearing_deg, ranges, err_deg, recv_rad,
866             switched=switched)
867         sampled_coverage = res["coverage"]
868         feasible = guaranteed & (sampled_coverage + eps >= min_coverage)
869         safe_vals = vals.copy()
870         safe_vals[~feasible] = math.inf
871         res["guaranteed"] = guaranteed
872         res["pareto"] = pareto_mask(safe_vals)
873         return res

```

B.3 问题三代码

编号 13 七点扫描骨架 (q3_scan_sites, 对应论文五 §3)

```

35 def q3_scan_sites(radius: float = 1200.0) -> Arr:
36     angle = np.arange(6) * math.pi / 3.0
37     ring = radius * np.column_stack((np.cos(angle), np.sin(angle)))
38     return np.vstack((np.zeros(2), ring))

```

编号 14 保证有信号的第二观测点 (safe_lateral_site, 对应论文五 §4.1)

```

57 def safe_lateral_site(site: Arr, bearing_deg: float, side: float = 1.0)
    -> Arr:
58     angle = math.radians(bearing_deg)
59     u = np.array([math.cos(angle), math.sin(angle)])
60     v = np.array([-math.sin(angle), math.cos(angle)])
61     return site + 400.0 * u + side * 300.0 * v

```

编号 15 定位区域与最小覆盖圆 (Q3Agent._region, 对应论文五 §4.1)

```

121 def _region(self, track: ChannelTrack) -> tuple[Arr, Arr, float]:
122     sites = np.vstack([obs.site for obs in track.obs])
123     bearings = np.array([obs.bearing_deg for obs in track.obs])
124     poly, _ = localization_polygon(
125         sites, bearings, err_deg=self.err_deg, n_bound=1440, outer=True)
126     if len(poly) == 0:
127         track.status = "geometry_fault"
128         raise ApiError(f"empty bearing intersection on channel {track.
129             channel}")
129     center, radius = minimum_enclosing_circle(poly)
130     return poly, center, radius

```

编号 16 指令发送与幂等重试 (ApiClient._post, 对应论文五 §2)

```

95 def _post(self, path: str, payload: Json) -> Json:
96     with self.lock:
97         self._check_deadline(path)
98         saw_response = False

```

```

99     last_status = 0
100     for attempt in range(self.max_retry + 1):
101         try:
102             status, body = self._post_once(path, payload)
103             saw_response = True
104             last_status = status
105             if status == 200 and body.get("accepted") is True:
106                 return body
107             if status not in (429, 500):
108                 detail = body.get("error") or body.get("message") or body.
get("raw")
109                 extra = f", detail={detail}" if detail else ""
110                 hint = ""
111                 if path == "/enter" and status == 200:
112                     hint = " (重复 /enter 会被拒绝, 每局只允许进入一次)"
113                 raise ApiError(
114                     f"{path} 被拒绝: status={status}, "
115                     f"accepted={body.get('accepted')}{extra}{hint}", rejected=
True)
116         except (TimeoutError, ConnectionError, URLError,
117                 json.JSONDecodeError):
118             pass
119         if attempt == self.max_retry:
120             break
121         self._check_deadline(path)
122         time.sleep(min(self.backoff_s * 2 ** attempt, 1.0))
123         if saw_response:
124             raise ApiError(
125                 f"{path} 重试 {self.max_retry + 1} 次仍被拒绝: status={
last_status}",
126                 rejected=True)
127             raise ApiError(
128                 f"{path} 连接失败(共 {self.max_retry + 1} 次): 接口可能未开放或
已结束")

```

编号 17 等待接口开放 (ApiClient.enter_when_open, 对应论文五 §2)

```

139 def enter_when_open(self, wait_s: float = 300.0,
140                     request_id: str = "enter-wait-000001") -> Json:
141     payload = self._base(request_id)
142     end = time.monotonic() + wait_s
143     delay = 0.5
144     rejected: ApiError | None = None
145     while True:
146         try:
147             return self.enter_payload(payload)
148         except ApiError as exc:
149             if exc.rejected:

```

```

150         rejected = exc
151         break
152     except (TimeoutError, ConnectionError, URLError):
153         pass
154     if time.monotonic() + delay >= end:
155         raise ApiError(
156             f"接口在 {wait_s:.0f} 秒内未开放，请确认已开始测试并等完 5 秒
157             倒计时")
158         time.sleep(delay)
159         delay = min(delay * 1.5, 2.0)
160     raise ApiError(
161         f"/enter 被拒绝: {rejected}. 请检查本局是否已被进入过，"
162         "以及是否还有另一个程序在运行")

```

编号 18 单频道定位清除与兜底 (Q3Agent._finish_channel, 对应论文五 §4.2)

```

158 def _finish_channel(self, track: ChannelTrack) -> None:
159     last_poly: Arr | None = None
160     for _ in range(self.max_obs):
161         if track.status == "cleared":
162             return
163         if len(track.obs) == 1:
164             center = track.obs[0].site
165             radius = math.inf
166         else:
167             poly, center, radius = self._region(track)
168             last_poly = poly
169             if radius <= 20.0:
170                 track.status = "clear_ready"
171                 self._clear(center, track)
172                 return
173             track.radii.append(radius)
174             site = self._next_site(track, center, radius)
175             if self._seen(track, site):
176                 track.status = "geometry_fault"
177                 raise ApiError(f"repeated localization site on channel {track.
178                 channel}")
179             body = self._measure(site, track.channel)
180             result = body.get("measure_result")
181             if result == "no_signal":
182                 self._lost_signal(track)
183                 return
184             self._record(track, site, body)
185             track.status = "geometry_fault"
186             detail = "after intersection" if last_poly is not None else "before
187             intersection"
188             raise ApiError(
189                 f"measurement cap reached {detail} on channel {track.channel}")

```

编号 19 扫描发现、批量调度与终止条件 (Q3Agent.run / _discover、_finish_pending, 对应论文文五 §5、§6.2)

```
241 def run(self, enter_body: dict[str, object] | None = None) ->
    MissionResult:
242     if enter_body is None:
243         enter_body = self.client.enter()
244         self.entered_here = True
245     self.virtual_s = float(enter_body["virtual_time_s"])
246     try:
247         self._discover(immediate=self.schedule == "immediate")
248         if self.schedule == "batch":
249             self._finish_pending()
250             self._certify_unknown()
251             complete = self.cleared_mask | self.absent_mask == (1 << 20) - 1
252             if not complete:
253                 raise ApiError("mission ended without a channel certificate")
254             out = self.client.exit()
255             self.virtual_s = float(out["virtual_time_s"])
256     except Exception:
257         try:
258             self.client.exit()
259         except Exception:
260             pass
261         raise
262     cleared = sum(track.status == "cleared" for track in self.tracks.
    values())
263     absent = sum(track.status == "absent_certified" for track in self.
    tracks.values())
264     return MissionResult(
265         cleared, absent, self.measures, self.clear_calls,
266         self.virtual_s, complete)
```

```
202 def _discover(self, immediate: bool) -> None:
203     stop = False
204     for site in self._scan_sites():
205         for channel in self._channel_order():
206             track = self.tracks[channel]
207             body = self._measure(site, channel)
208             if body.get("measure_result") == "no_signal":
209                 continue
210             self._record(track, site, body)
211             if immediate and track.status != "cleared":
212                 self._finish_channel(track)
213             if self._known_count() >= 16:
214                 stop = True
```

```

215         break
216     if stop:
217         break
218
219 def _finish_pending(self) -> None:
220     while True:
221         pending = [
222             track for track in self.tracks.values()
223             if track.status in {"positively_detected", "localizing"}
224         ]
225         if not pending:
226             return
227         def travel(track: ChannelTrack) -> float:
228             first = track.obs[0]
229             site = safe_lateral_site(first.site, first.bearing_deg)
230             return float(np.linalg.norm(site - self.pos))
231
232         track = min(pending, key=lambda item: (travel(item), item.channel)
233     )
234     self._finish_channel(track)

```

编号 19a 频道处理顺序 (Q3Agent._channel_order, 对应论文五 §6.2)

```

195 def _channel_order(self) -> list[int]:
196     unknown = [
197         channel for channel, track in self.tracks.items()
198         if track.status == "unknown"
199     ]
200     return sorted(unknown, key=lambda channel: (channel - self.
current_channel) % 20)

```

编号 23 联合选路的精确解、2-opt / Or-opt 与主入口 (exact_open / improve_open / joint_route, 对应论文五 §5)

```

259 def exact_open(distances: Arr) -> list[int]:
260     n = len(distances) - 1
261     dp = np.full((1 << n, n), math.inf)
262     parent = np.full((1 << n, n), -1, dtype=np.int16)
263     for i in range(n):
264         dp[1 << i, i] = distances[n, i]
265     for mask in range(1, 1 << n):
266         for last in range(n):
267             if not mask & (1 << last) or not math.isfinite(float(dp[mask, last]
)):
268                 continue
269         for nxt in range(n):
270             if mask & (1 << nxt):
271                 continue
272         new_mask = mask | (1 << nxt)

```

```

273     value = float(dp[mask, last] + distances[last, nxt])
274     if value < dp[new_mask, nxt]:
275         dp[new_mask, nxt] = value
276         parent[new_mask, nxt] = last
277     mask = (1 << n) - 1
278     last = int(np.argmin(dp[mask]))
279     order = []
280     while last >= 0:
281         order.append(last)
282         previous = int(parent[mask, last])
283         mask ^= 1 << last
284         last = previous
285     return order[::-1]

```

```

287 def improve_open(order: list[int], distances: Arr) -> list[int]:
288     out = order.copy()
289     n = len(out)
290     start = n
291     for _ in range(8):
292         changed = False
293         for i in range(n - 1):
294             a = start if i == 0 else out[i - 1]
295             for j in range(i + 1, n):
296                 b, c = out[i], out[j]
297                 before = float(distances[a, b])
298                 after = float(distances[a, c])
299                 if j + 1 < n:
300                     d = out[j + 1]
301                     before += float(distances[c, d])
302                     after += float(distances[b, d])
303                 if after + 1e-6 < before:
304                     out[i:j + 1] = out[i:j + 1][::-1]
305                     changed = True
306     best_cost = route_cost(out, distances)
307     best = None
308     for i in range(n):
309         reduced = out[:i] + out[i + 1:]
310         for j in range(n):
311             candidate = reduced[:j] + [out[i]] + reduced[j:]
312             value = route_cost(candidate, distances)
313             if value + 1e-6 < best_cost:
314                 best_cost, best = value, candidate
315     if best is not None:
316         out = best
317         changed = True
318     if not changed:
319         break

```

```

320     return out

322 def joint_route(points: Arr, start: Arr, station_count: int) -> list[int]:
323     n = len(points)
324     if n == 0:
325         return []
326     p = np.vstack((points, start))
327     distances = np.linalg.norm(p[:, None, :] - p[None, :, :], axis=2)
328     if n <= 10:
329         return exact_open(distances)
330     incumbent = list(range(station_count))
331     for i in range(station_count, n):
332         candidates = [incumbent[:j] + [i] + incumbent[j:]]
333                     for j in range(len(incumbent) + 1)]
334         incumbent = min(candidates, key=lambda order: route_cost(order,
335                             distances))
336     seeds = [incumbent]
337     for first in np.argsort(distances[n, :n])[:3]:
338         order = [int(first)]
339         left = set(range(n)) - {int(first)}
340         while left:
341             i = min(left, key=lambda j: (float(distances[order[-1], j]), j))
342             order.append(i)
343             left.remove(i)
344         seeds.append(order)
345     best = incumbent
346     best_cost = route_cost(best, distances)
347     for seed in seeds:
348         order = improve_open(seed, distances)
349         value = route_cost(order, distances)
350         if value + 1e-6 < best_cost:
351             best, best_cost = order, value
352     if sorted(best) != list(range(n)):
353         raise ValueError("route lost a required node")
354     return best

```

编号24 阴性证据的连续域覆盖推理(covered_by_negative_disks / unknown_negative_implied, 对应论文五 §3)

```

209 def covered_by_negative_disks(poly: Arr, negatives: Arr) -> bool:
210     stack = [(poly, 0)]
211     count = 0
212     while stack:
213         cell, depth = stack.pop()
214         count += 1
215         far = np.max(np.linalg.norm(cell[:, None, :] - negatives[None, :,
216                                     :], axis=2),

```

```

216         axis=0)
217     if np.any(far <= 999.99):
218         continue
219     if depth >= 9 or count >= 80:
220         return False
221     center = cell.mean(axis=0)
222     if np.min(np.linalg.norm(negatives - center, axis=1)) >= 999.99:
223         return False
224     horizontal = np.ptp(cell[:, 0]) >= np.ptp(cell[:, 1])
225     axis = np.array([1.0, 0.0]) if horizontal else np.array([0.0, 1.0])
226     projection = cell @ axis
227     middle = (float(projection.min()) + float(projection.max())) / 2.0
228     for normal, limit in ((axis, middle), (-axis, -middle)):
229         part = cut(cell, normal, limit)
230         if len(part):
231             stack.append((part, depth + 1))
232     return True

```

```

234 def unknown_negative_implied(site: Arr, negatives: list[Arr]) -> bool:
235     if not negatives:
236         return False
237     old = np.asarray(negatives)
238     if np.min(np.linalg.norm(old - site, axis=1)) <= 1e-7:
239         return True
240     if len(old) < 2:
241         return False
242     poly = cut_disk(regular_bound(1800.0, 72, outer=True), site, 1500.0)
243     for previous in old:
244         delta = previous - site
245         limit = float(delta @ site + 0.5 * (delta @ delta))
246         poly = cut(poly, delta, limit)
247         if len(poly) == 0:
248             return True
249     return covered_by_negative_disks(poly, old)

```

编号 25 分箱包围的测站定位半径上界 (bearing_radius_bound, 对应论文五 §4.1)

```

355 def bearing_radius_bound(poly: Arr, site: Arr, err: float = 1.01,
356                          step: float = 2.0) -> float:
357     if not 0 < step <= 10 or not 0 < err < 10:
358         raise ValueError("invalid bearing enclosure parameters")
359     if len(poly) == 0 or not np.all(np.isfinite(poly)) or not np.all(np.
360                                isfinite(site)):
361         raise ValueError("invalid bearing enclosure geometry")
362     delta = poly - site
363     angles = np.sort(np.mod(np.arctan2(delta[:, 1], delta[:, 0]), 2 * math
364                                .pi))
365     gaps = np.diff(np.r_[angles, angles[0] + 2 * math.pi])

```

```

364 k = int(np.argmax(gaps))
365 span = 2 * math.pi - float(gaps[k])
366 if span < math.pi - 1e-8 and np.min(np.linalg.norm(delta, axis=1)) > 1
    e-7:
367     lo = math.degrees(float(angles[(k + 1) % len(angles)])) - err - 1e-7
368     hi = lo + math.degrees(span) + 2 * err + 2e-7
369 else:
370     lo, hi = 0.0, 360.0
371 count = max(1, math.ceil((hi - lo) / step))
372 width = (hi - lo) / count
373 worst = 0.0
374 for i in range(count):
375     angle = lo + (i + 0.5) * width
376     outer = cut_bearing(poly, site, angle, err + width / 2 + 1e-7)
377     if len(outer):
378         worst = max(worst, enclosing(outer)[1])
379 return worst + 1e-3

```

编号27 小半径方向完备测站包与误差上界(packet_sites / packet_localization_bound, 对应论文五 §4.2 / 六 §4.2)

```

406 def packet_sites(center: Arr, radius: float, start: Arr) -> Arr:
407     if not 0 <= radius <= 300.0:
408         raise ValueError("packet radius exceeds its receiving certificate")
409     size = 1.2 * radius + 8.0
410     phase = math.atan2(start[1] - center[1], start[0] - center[0])
411     angles = phase + np.arange(12) * math.pi / 6.0
412     return center + size * np.column_stack((np.cos(angles), np.sin(angles)
    ))

```

```

414 def packet_localization_bound(radius: float, err: float = 1.01) -> float
    :
415     size = 1.2 * radius + 8.0
416     h, w = size * math.cos(math.pi / 12), size * math.sin(math.pi / 12)
417     low = 2 * math.atan2(w, h + radius)
418     high = 2 * math.atan2(w, h - radius)
419     error = math.radians(err)
420     gap = min(low - 2 * error, math.pi - high - 2 * error)
421     if gap <= 0:
422         return math.inf
423     factor = math.sin(error) / math.sin(gap / 2)
424     if factor >= 1:
425         return math.inf
426     return 2 * factor * (size + radius) / (1 - factor) + 1e-3

```

编号28 顺路收获、站内排程与联合选路入口(_opportunity_jobs / _harvest、scan_station / _plan, 对应论文五 §5、§6.2)

```

459 def _opportunity_jobs(self, site: Arr, excluded: int = 0) -> list[
    FastTrack]:
460     if not self.opportunistic:
461         return []
462     jobs = []
463     future = [self.sites[i] for i in self.future
464               if i in self.todo and np.linalg.norm(self.sites[i] - site)
    > 1e-7]
465     for track in self.tracks.values():
466         if (track.channel == excluded or track.center is None or track.
    poly is None
467             or track.status in {"unknown", "cleared", "absent_certified"}
468             or track.radius <= 19.999 or track.opportunities >= 5
469             or self._cached(track, site) is not None or self.
    _known_negative(track, site)):
470             continue
471         pred = self._forecast(track, site)
472         saving = 2.0 * max(0.0, track.radius - pred) / 5.0
473         cost = 5.0 + float(track.channel != self.current_channel) + float(
    excluded != 0)
474         if pred > 19.5 and saving < 1.5 * cost:
475             continue
476         if pred > 19.5 and pred >= 0.68 * track.radius:
477             continue
478         if excluded and pred > 19.5 and pred >= 0.35 * track.radius:
479             continue
480         defer = False
481         for later in future[:3]:
482             if self._cached(track, later) is not None or self.
    _known_negative(track, later):
483                 continue
484             if (np.linalg.norm(later - track.center) + 100.0
    < np.linalg.norm(site - track.center)
485                 and self._forecast(track, later) <= pred + 1.0):
486                 defer = True
487                 break
488         if not defer:
489             jobs.append(track)
490     return jobs

```

```

507 def _harvest(self, site: Arr, excluded: int = 0) -> None:
508     for track in self._order_jobs(self._opportunity_jobs(site, excluded)
    ):
509         track.opportunities += 1
510         self._sense(track, site)
511         self._clear_here(track, site)
512

```

```

513 def _scan_station(self, index: int) -> None:
514     site = self.sites[index]
515     bit = 1 << index
516     mandatory = [t for t in self.tracks.values()
517                  if t.status == "unknown" and not t.station_mask & bit]
518     optional = self._opportunity_jobs(site)
519     required_channels = {t.channel for t in mandatory}
520     for track in self._order_jobs(mandatory + optional):
521         if track.status in {"cleared", "absent_certified"}:
522             continue
523         if track.channel not in required_channels:
524             track.opportunities += 1
525         body = self._sense(track, site)
526         if track.channel in required_channels and body.get("measure_result") == "no_signal":
527             track.station_mask |= bit
528             self._clear_here(track, site)
529             self._certify()

```

```

531 def _plan(self) -> tuple[str, int] | None:
532     self._retire_stations()
533     indices = [i for i in self.future if i in self.todo]
534     indices.extend(i for i in sorted(self.todo) if i not in indices)
535     nodes: list[tuple[str, int]] = [("scan", i) for i in indices]
536     points = [self.sites[i] for i in indices]
537     for track in self.tracks.values():
538         if (track.center is None
539             or track.status in {"unknown", "cleared", "absent_certified"}):
540             continue
541         if self.todo and (not self.inline or track.radius > 300.0):
542             continue
543         if self.todo and track.radius > 19.999 and any(
544             self._cached(track, self.sites[i]) is None
545             and not self._known_negative(track, self.sites[i])
546             and self._forecast(track, self.sites[i]) <= 19.5
547             and np.linalg.norm(self.sites[i] - track.center) + 100.0
548             < np.linalg.norm(self.pos - track.center) for i in indices
549             [:3]):
550             continue
551         nodes.append(("service", track.channel))
552         points.append(track.center)
553     if not nodes:
554         return None
555     order = joint_route(np.asarray(points), self.pos, len(indices))
556     self.future = [nodes[i][1] for i in order if nodes[i][0] == "scan"]
557     self.next_channel = next((nodes[i][1] for i in order if nodes[i][0]

```

557

```

== "service"), 0)
return nodes[order[0]]

```

B.4 问题四代码

编号 20 31 点等边三角格扫描集 (q4_scan_sites, 对应论文六 §3.2)

```

38 def q4_scan_sites(spacing: float = 990.0,
39                  target_rad: float = 1800.0) -> Arr:
40     if spacing <= 0 or spacing >= 1000.0 or target_rad <= 0:
41         raise ValueError("invalid Q4 lattice parameters")
42     e1 = np.array([spacing, 0.0])
43     e2 = np.array([spacing / 2.0, math.sqrt(3.0) * spacing / 2.0])
44     cutoff = target_rad + spacing
45     limit = int(math.ceil(cutoff / spacing)) * 2 + 2
46     points = []
47     for i in range(-limit, limit + 1):
48         for j in range(-limit, limit + 1):
49             point = i * e1 + j * e2
50             if np.linalg.norm(point) <= cutoff + 1e-9:
51                 points.append(point)
52     return ordered_route(np.asarray(points), np.zeros(2))

```

编号 21 定位区域与最小覆盖圆 (与问题三共用) (Q3Agent._region, 对应论文六 §4.2)

```

121 def _region(self, track: ChannelTrack) -> tuple[Arr, Arr, float]:
122     sites = np.vstack([obs.site for obs in track.obs])
123     bearings = np.array([obs.bearing_deg for obs in track.obs])
124     poly, _ = localization_polygon(
125         sites, bearings, err_deg=self.err_deg, n_bound=1440, outer=True)
126     if len(poly) == 0:
127         track.status = "geometry_fault"
128         raise ApiError(f"empty bearing intersection on channel {track.
129 channel}")
129     center, radius = minimum_enclosing_circle(poly)
130     return poly, center, radius

```

编号 22 扇形 20 m 覆盖清除路径与访问顺序 (bearing_clear_sites / ordered_route, 对应论文六 §4.2)

```

71 def bearing_clear_sites(
72     site: Arr, bearing_deg: float, err_deg: float = 1.01,
73     ranges: tuple[float, float] = (5.0, 1500.0),
74     cover_rad: float = 20.0, start: Arr | None = None,
75 ) -> Arr:
76     origin = np.asarray(site, dtype=float)
77     if origin.shape != (2,) or not np.all(np.isfinite(origin)):
78         raise ValueError("site must be a finite point")
79     lo, hi = ranges

```

```

80 if not 0 <= lo < hi or not 0 <= err_deg < 90 or cover_rad <= 0:
81     raise ValueError("invalid bearing sector parameters")
82 spacing = cover_rad * math.sqrt(3.0) * 0.995
83 e1 = np.array([spacing, 0.0])
84 e2 = np.array([spacing / 2.0, math.sqrt(3.0) * spacing / 2.0])
85 height = float(e2[1])
86 limit_i = int(math.ceil((hi + cover_rad) / spacing)) + 4
87 limit_j = int(math.ceil((hi * math.sin(math.radians(err_deg))
88                     + cover_rad) / height)) + 4
89 local = []
90 for i in range(-limit_i, limit_i + 1):
91     for j in range(-limit_j, limit_j + 1):
92         point = i * e1 + j * e2
93         local.append(point)
94 local_points = np.asarray(local)
95 keep = _distance_to_sector(
96     local_points, lo, hi, math.radians(err_deg)) <= cover_rad + 1e-9
97 local_points = local_points[keep]
98 angle = math.radians(bearing_deg)
99 rotation = np.array([
100     [math.cos(angle), -math.sin(angle)],
101     [math.sin(angle), math.cos(angle)],
102 ])
103 global_points = origin + local_points @ rotation.T
104 route_start = origin if start is None else np.asarray(start, dtype=
105     float)
106 return ordered_route(global_points, route_start)

```

```

10 def ordered_route(points: Arr, start: Arr) -> Arr:
11     p = np.asarray(points, dtype=float)
12     if p.ndim != 2 or p.shape[1] != 2 or len(p) == 0:
13         raise ValueError("points must have shape (n, 2) with n positive")
14     current = np.asarray(start, dtype=float)
15     unused = list(range(len(p)))
16     order: list[int] = []
17     while unused:
18         pick = min(unused, key=lambda i: (float(np.linalg.norm(p[i] -
19             current)), i))
20         order.append(pick)
21         unused.remove(pick)
22         current = p[pick]
23     route = p[order].copy()
24     improved = True
25     while improved:
26         improved = False
27         for i in range(1, len(route) - 1):
28             for j in range(i + 1, len(route)):

```

```

28     before = float(np.linalg.norm(route[i - 1] - route[i]))
29     after = float(np.linalg.norm(route[i - 1] - route[j]))
30     if j + 1 < len(route):
31         before += float(np.linalg.norm(route[j] - route[j + 1]))
32         after += float(np.linalg.norm(route[i] - route[j + 1]))
33     if after + 1e-9 < before:
34         route[i:j + 1] = route[i:j + 1][::-1]
35         improved = True
36     return route

```

编号 22a 问题三与问题四在证据解释处的分支 (Q3Agent._lost_signal, 对应论文六 §5)

```

154 def _lost_signal(self, track: ChannelTrack) -> None:
155     track.status = "geometry_fault"
156     raise ApiError(f"guaranteed follow-up lost signal on channel {track.
channel}")

```

编号 26 22 点集与连续域方向完备认证 (direction_certificate / compact_q4_sites, 对应论文六 §3.3)

```

177 def direction_certificate(sites: Arr) -> tuple[bool, int]:
178     boundary = regular_bound(1800.0, 72, outer=True)
179     stack = [(np.vstack((np.zeros(2), boundary[i], boundary[(i + 1) % 72])), 0)
180             for i in range(72)]
181     count = 0
182     while stack:
183         cell, depth = stack.pop()
184         count += 1
185         distances = np.linalg.norm(cell[:, None, :] - sites[None, :, :],
axis=2)
186         eligible = sites[np.max(distances, axis=0) <= 994.99]
187         if len(eligible) >= 3:
188             hull = convex_hull(eligible, tol=0.0)
189             if inside_hull(hull, cell):
190                 continue
191             if depth >= 22 or count >= 60000:
192                 return False, count
193             lengths = np.linalg.norm(np.roll(cell, -1, axis=0) - cell, axis=1)
194             i = int(np.argmax(lengths))
195             j, k = (i + 1) % 3, (i + 2) % 3
196             middle = (cell[i] + cell[j]) / 2.0
197             stack.append((np.vstack((cell[i], middle, cell[k])), depth + 1))
198             stack.append((np.vstack((middle, cell[j], cell[k])), depth + 1))
199     return True, count

```

```

201 def compact_q4_sites() -> Arr:
202     inner_r, outer_r = 975.0, 1850.0
203     a = np.arange(7) * (2 * math.pi / 7)

```

```

204 b = np.arange(14) * (2 * math.pi / 14)
205 inner = inner_r * np.column_stack((np.cos(a), np.sin(a)))
206 outer = outer_r * np.column_stack((np.cos(b), np.sin(b)))
207 return np.vstack((np.zeros(2), inner, outer[np.r_[12, 13, np.arange
(12)]]))

```

编号27 小半径方向完备测站包与误差上界(packet_sites / packet_localization_bound, 对应论文五 §4.2 / 六 §4.2)

```

406 def packet_sites(center: Arr, radius: float, start: Arr) -> Arr:
407     if not 0 <= radius <= 300.0:
408         raise ValueError("packet radius exceeds its receiving certificate")
409     size = 1.2 * radius + 8.0
410     phase = math.atan2(start[1] - center[1], start[0] - center[0])
411     angles = phase + np.arange(12) * math.pi / 6.0
412     return center + size * np.column_stack((np.cos(angles), np.sin(angles)
))

```

```

414 def packet_localization_bound(radius: float, err: float = 1.01) -> float
:
415     size = 1.2 * radius + 8.0
416     h, w = size * math.cos(math.pi / 12), size * math.sin(math.pi / 12)
417     low = 2 * math.atan2(w, h + radius)
418     high = 2 * math.atan2(w, h - radius)
419     error = math.radians(err)
420     gap = min(low - 2 * error, math.pi - high - 2 * error)
421     if gap <= 0:
422         return math.inf
423     factor = math.sin(error) / math.sin(gap / 2)
424     if factor >= 1:
425         return math.inf
426     return 2 * factor * (size + radius) / (1 - factor) + 1e-3

```

编号29 空清除预算、最后单元覆盖证明与近距包收尾(Q4FastAgentV3._clear / _cover_fits / _finish_cover, 对应论文六 §4.2)

```

157 def _clear(self, track: FastTrack, site: Arr, near: bool = False,
158           certified: bool = True) -> bool:
159     if track.status == "cleared":
160         return True
161     if not certified and self.empty_calls >= self.empty_limit:
162         if track.poly is None or not track.obs:
163             raise ApiError("missing last-cell certificate")
164         angle = math.radians(track.obs[0].bearing_deg)
165         cells = cover_cells(track.poly, 19.5, angle)
166         disks = track.failures + [site]
167         if not all(any(np.max(np.linalg.norm(cell - p, axis=1)) <= 19.999
168                        for p in disks) for cell in cells):
169             raise ApiError("clear would exceed the mission-wide budget")

```

```

170     ok = super()._clear(track, site, near, certified)
171     if not ok:
172         self.empty_calls += 1
173         if self.empty_calls > self.empty_limit:
174             track.status = "geometry_fault"
175             raise ApiError("last-cell clear contradicted its coverage
certificate")
176     return ok

```

```

178     def _cover_fits(self, track: FastTrack) -> bool:
179         if track.poly is None or not track.obs:
180             return False
181         angle = math.radians(track.obs[0].bearing_deg)
182         count = len(cover_cells(track.poly, 19.5, angle))
183         return count - 1 <= self.empty_limit - self.empty_calls

```

```

196     def _finish_cover(self, track: FastTrack) -> None:
197         if self._ready_clear(track):
198             return
199         if self._cover_fits(track):
200             before, self.phase = self.phase, "budgeted_clear_cover"
201             self.branches["budgeted_clear_cover"] += 1
202             try:
203                 Q4FastAgent._finish_cover(self, track)
204             finally:
205                 self.phase = before
206             return
207         if track.poly is None:
208             raise ApiError("missing directional packet-cover region")
209         before, self.phase = self.phase, "directional_packet_cover"
210         self.branches["directional_packet_cover"] += 1
211         try:
212             angle = math.radians(track.obs[0].bearing_deg)
213             radius = 19.5
214             while packet_localization_bound(radius, self.err_deg) > 19.0:
215                 radius /= 2
216                 if radius < 0.01:
217                     raise ApiError("no finite packet localization certificate")
218             cells = cover_cells(track.poly, radius, angle)
219             centers = np.vstack([enclosing(cell)[0] for cell in cells])
220             left = set(range(len(cells)))
221             while left:
222                 if self._ready_clear(track):
223                     return
224                 i = min(left, key=lambda j: (float(np.linalg.norm(centers[j] -
self.pos)), j))
225                 left.remove(i)

```

```
226         if definitely_disjoint(cells[i], track.poly):
227             continue
228         sites = packet_sites(centers[i], radius, self.pos)
229         for site in sites:
230             self._sense(track, site)
231             if self._ready_clear(track):
232                 return
233         track.status = "geometry_fault"
234         raise ApiError("certified directional packet coverage exhausted")
235     finally:
236         self.phase = before
```